



# Lab Mid

OOP

Instructor: Mr. Mohsin Ahmed

|                         |                      |
|-------------------------|----------------------|
| <b>Student Name:</b>    | Muhammad Abdullah    |
| <b>Registration No:</b> | FA24-BSE-042         |
| <b>Department:</b>      | Software Engineering |
| <b>Submission Date:</b> | 22th October 2022    |

COMSATS University Islamabad

## Answer #1

### Code:

```
package Lab_Mid;

abstract class Vehicle {
    private String vehicleNumber;
    private String ownerName;

    public Vehicle(String vehicleNumber, String ownerName) {
        this.vehicleNumber = vehicleNumber;
        this.ownerName = ownerName;
    }

    public String getVehicleNumber() {
        return vehicleNumber;
    }

    public String getOwnerName() {
        return ownerName;
    }

    public abstract double calculateParkingFee(int hours);
}

class Car extends Vehicle {
    private static final double FIRST_HOUR_FEE = 50;
    private static final double PER_HOUR_RATE = 30;

    public Car(String vehicleNumber, String ownerName) {
        super(vehicleNumber, ownerName);
    }

    @Override
    public double calculateParkingFee(int hours) {
        if (hours <= 1) return FIRST_HOUR_FEE;
        return FIRST_HOUR_FEE + (hours - 1) * PER_HOUR_RATE;
    }
}

class Bike extends Vehicle {
    private static final double FIRST_HOUR_FEE = 20;
    private static final double PER_HOUR_RATE = 10;

    public Bike(String vehicleNumber, String ownerName) {
        super(vehicleNumber, ownerName);
    }
}
```

```

@Override
public double calculateParkingFee(int hours) {
    if (hours <= 1) return FIRST_HOUR_FEE;
    return FIRST_HOUR_FEE + (hours - 1) * PER_HOUR_RATE;
}
}

class Truck extends Vehicle {
    private static final double FIRST_HOUR_FEE = 100;
    private static final double PER_HOUR_RATE = 70;

    public Truck(String vehicleNumber, String ownerName) {
        super(vehicleNumber, ownerName);
    }

    @Override
    public double calculateParkingFee(int hours) {
        if (hours <= 1) return FIRST_HOUR_FEE;
        return FIRST_HOUR_FEE + (hours - 1) * PER_HOUR_RATE;
    }
}

class ParkingSlot {
    private Vehicle vehicle;

    public ParkingSlot(Vehicle vehicle) {
        this.vehicle = vehicle;
    }

    public void showParkingDetails(int hours) {
        System.out.println("Vehicle No: " + vehicle.getVehicleNumber());
        System.out.println("Owner: " + vehicle.getOwnerName());
        System.out.println("Total Fee: $" + vehicle.calculateParkingFee(hours));
        System.out.println("-----");
    }
}

public class SmartParkingRunner {
    public static void main(String[] args) {
        ParkingSlot carSlot = new ParkingSlot(new Car("CAR-2025", "Abdullah"));
        ParkingSlot bikeSlot = new ParkingSlot(new Bike("BIKE-909", "Ali"));
        ParkingSlot truckSlot = new ParkingSlot(new Truck("TRUCK-777", "Zain"));

        carSlot.showParkingDetails(3);
        bikeSlot.showParkingDetails(2);
        truckSlot.showParkingDetails(5);
    }
}

```

## Output:

```
mabdullah@MacBook Lab_Mid % java SmartParkingRunner.java
Vehicle No: CAR-2025
Owner: Abdullah
Total Fee: $110.0
-----
Vehicle No: BIKE-909
Owner: Ali
Total Fee: $30.0
-----
Vehicle No: TRUCK-777
Owner: Zain
Total Fee: $380.0
-----
```

## Summary:

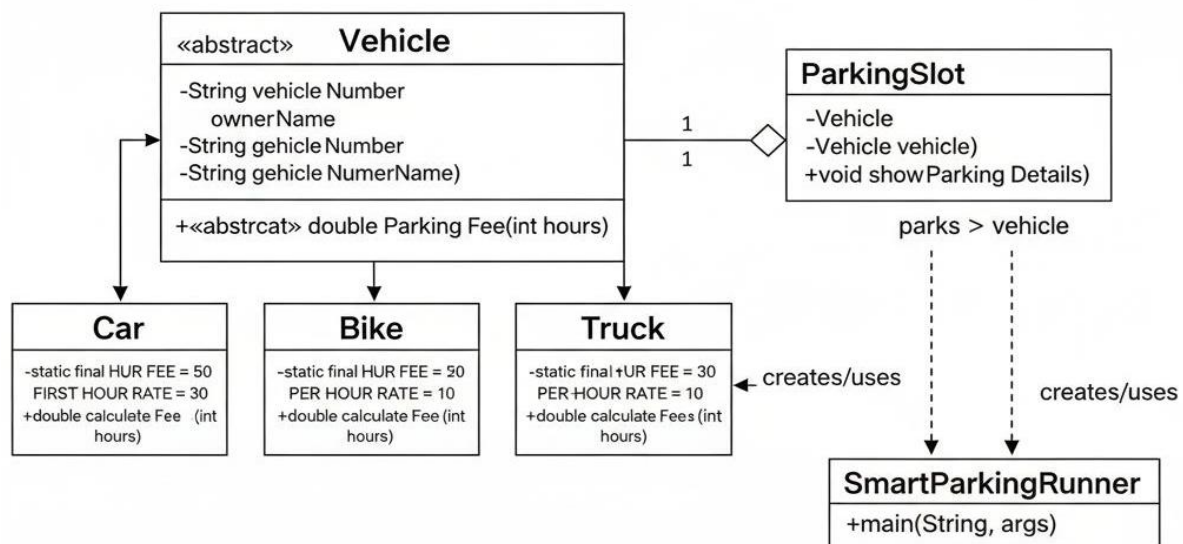
**Inheritance:** Car, Bike, and Truck extend the abstract base class Vehicle. This means they share the common fields and methods of Vehicle.

**Polymorphism:** All subclasses provide their specific implementation for the abstract method calculateParkingFee(int hours), which is the core difference between the vehicle types.

**Composition:** ParkingSlot holds a single Vehicle reference. The ParkingSlot doesn't care which specific type of vehicle it holds; it relies on the generic Vehicle interface (Polymorphism) to call calculateParkingFee().

**Abstraction:** Vehicle is marked as an abstract class (<<abstract>>) with an abstract method, enforcing the calculateParkingFee contract for all derived classes.

## Class Diagram:



## Answer #2

### Code:

```
package Lab_Mid;

import java.util.*;

abstract class SmartDevice {
    private String deviceId;
    private String deviceName;
    private boolean powerStatus;
    private String networkName;

    public SmartDevice(String deviceId, String deviceName) {
        this.deviceId = deviceId;
        this.deviceName = deviceName;
        this.powerStatus = false;
    }

    public String getDeviceId() {
        return deviceId;
    }
    public String getDeviceName() {
        return deviceName;
    }
    public boolean getPowerStatus() {
        return powerStatus;
    }

    public void connectToNetwork(String networkName) {
        this.networkName = networkName;
        System.out.println(deviceName + " connected to network: " + networkName);
    }

    public void turnOn() {
        powerStatus = true;
        System.out.println(deviceName + " is turned ON.");
    }

    public void turnOff() {
        powerStatus = false;
        System.out.println(deviceName + " is turned OFF.");
    }

    // Abstract method
    public abstract void showStatus();
}
```

```
class SmartLight extends SmartDevice {
    private int brightnessLevel;

    public SmartLight(String id, String name, int brightnessLevel) {
        super(id, name);
        this.brightnessLevel = brightnessLevel;
    }

    @Override
    public void turnOn() {
        super.turnOn();
        System.out.println("Light brightness set to " + brightnessLevel + "%");
    }

    @Override
    public void showStatus() {
        System.out.println(getDeviceName() + " | Light | Status: " + (getPowerStatus() ? "ON" : "OFF"));
    }
}

class SmartFan extends SmartDevice {
    private int speed;

    public SmartFan(String id, String name, int speed) {
        super(id, name);
        this.speed = speed;
    }

    @Override
    public void turnOn() {
        super.turnOn();
        System.out.println("Fan running at speed " + speed);
    }

    @Override
    public void showStatus() {
        System.out.println(getDeviceName() + " | Fan | Status: " + (getPowerStatus() ? "ON" : "OFF"));
    }
}

class SmartThermostat extends SmartDevice {
    private double temperature;

    public SmartThermostat(String id, String name, double temperature) {
        super(id, name);
        this.temperature = temperature;
    }
}
```

```

// Method overloading
public void setTemperature(double celsius) {
    this.temperature = celsius;
    System.out.println("Temperature set to " + celsius + "°C");
}

public void setTemperature(double value, boolean inFahrenheit) {
    if (inFahrenheit) {
        this.temperature = (value - 32) * 5 / 9;
        System.out.println("Temperature set to " + temperature + "°C (converted from Fahrenheit)");
    } else {
        setTemperature(value);
    }
}

@Override
public void turnOn() {
    super.turnOn();
    System.out.println("Thermostat set at " + temperature + "°C");
}

@Override
public void showStatus() {
    System.out.println(getDeviceName() + " | Thermostat | Status: " + (getPowerStatus() ? "ON" : "OFF"));
}
}

// Composition
class SmartHomeController {
    private List<SmartDevice> devices = new ArrayList<>();

    public void addDevice(SmartDevice device) {
        devices.add(device);
    }

    public void turnAllOn() {
        for (SmartDevice d : devices) d.turnOn();
    }

    public void turnAllOff() {
        for (SmartDevice d : devices) d.turnOff();
    }

    public void showAllStatus() {
        for (SmartDevice d : devices) d.showStatus();
    }
}

```

```

public class SmartHomeRunner {
    public static void main(String[] args) {
        SmartLight light = new SmartLight("L1", "Bedroom Light", 80);
        SmartFan fan = new SmartFan("F1", "Ceiling Fan", 3);
        SmartThermostat thermostat = new SmartThermostat("T1", "Room Thermostat", 22.5);

        light.connectToNetwork("Home_WiFi");
        fan.connectToNetwork("Home_WiFi");
        thermostat.connectToNetwork("Home_WiFi");

        SmartHomeController controller = new SmartHomeController();
        controller.addDevice(light);
        controller.addDevice(fan);
        controller.addDevice(thermostat);

        controller.turnAllOn();
        thermostat.setTemperature(75, true); // Overloaded method
        controller.showAllStatus();
        controller.turnAllOff();
    }
}

```

## Output:

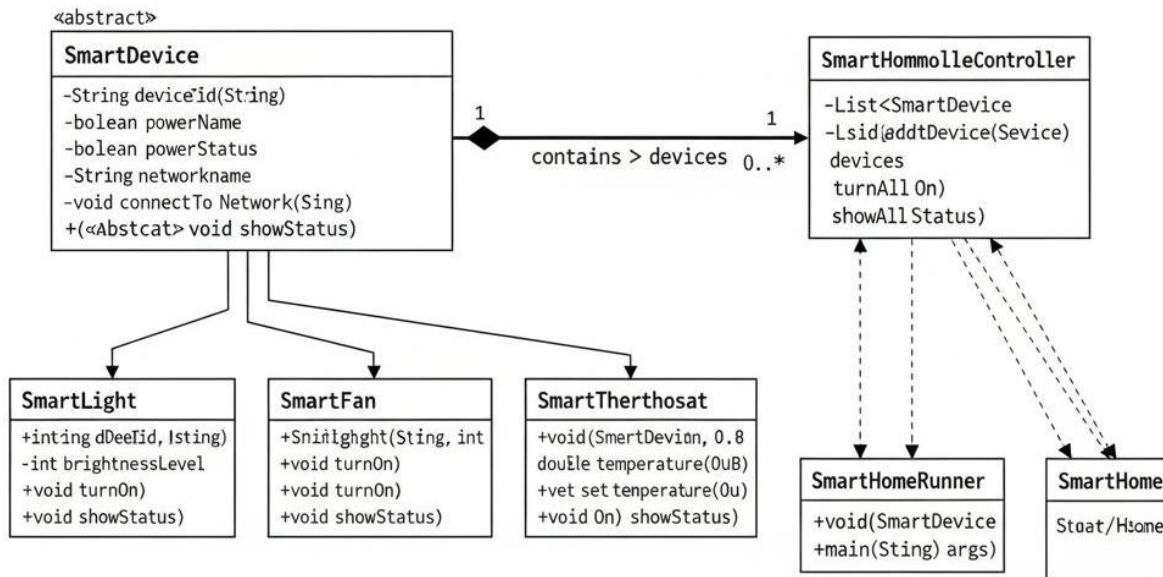
```

● mabdullah@MacBook Lab_Mid % java SmartHomeRunner.java
Bedroom Light connected to network: Home_WiFi
Ceiling Fan connected to network: Home_WiFi
Room Thermostat connected to network: Home_WiFi
Bedroom Light is turned ON.
Light brightness set to 80%
Ceiling Fan is turned ON.
Fan running at speed 3
Room Thermostat is turned ON.
Thermostat set at 22.5°C
Temperature set to 23.88888888888889°C (converted from Fahrenheit)
Bedroom Light | Light | Status: ON
Ceiling Fan | Fan | Status: ON
Room Thermostat | Thermostat | Status: ON
Bedroom Light is turned OFF.
Ceiling Fan is turned OFF.
Room Thermostat is turned OFF.

```

## Class Diagram:





## Summary:

**Reusability** is achieved via the abstract SmartDevice base class. It defines common behaviours like ( like turnOn, connectToNetwork, etc ) once, which are then inherited and reused by all subclasses ( SmartLight, SmartFan, etc ), preventing code deduplication.

**Flexibility** is provided by Polymorphism ( overriding & overloading ). **SmartHomeController** uses **composition** with the generic SmartDevice type.